
UNIVERSIDAD TECNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computacion y Diseno Digital
Carrera de Ingenieria de Software

BASES DE DATOS DISTRIBUIDAS

Material Academico Integral

Definicion, conceptos, características, arquitecturas,
fragmentacion, replicacion, transacciones, NoSQL y ejercicio practico

Asignatura: Aplicaciones Distribuidas (ISR-701)

Unidad: 3 – Bases de Datos Distribuidas

Semanas: 9, 10 y 11 del cronograma de 18 semanas

Periodo Academico: 2026–2027

Docente: Ph.D. Gleiston C. Guerrero-Ulloa

Resultado de Aprendizaje: Implementa soluciones de bases de datos distribuidas evaluando su rendimiento, consistencia y tolerancia a fallos.

Índice

1. Introduccion	3
2. Definicion Formal de Base de Datos Distribuida	3
3. Conceptos Relacionados con Bases de Datos Distribuidas	4
3.1. Sitio o Nodo	4
3.2. Esquema Global y Esquemas Locales	4
3.3. Fragmento	4
3.4. Replica	4
3.5. Catalogo Distribuido	4
3.6. Transaccion Distribuida	4
3.7. Procesamiento de Consultas Distribuido	4
3.8. Modelo de Consistencia	5
3.9. Tolerancia a Particiones de Red	5
4. Caracteristicas de las Bases de Datos Distribuidas	5
4.1. Distribucion Fisica y Logica	5
4.2. Autonomia Local	5
4.3. Crecimiento Incremental	5
4.4. Mayor Disponibilidad	5
4.5. Mejor Rendimiento Localizado	5
4.6. Heterogeneidad Potencial	6
4.7. Compromisos del Teorema CAP	6
5. Las 12 Reglas de Date para SGBDD	6
6. Arquitecturas de SGBDD	7
6.1. Arquitectura Cliente-Servidor	7
6.2. Arquitectura Federada	7
6.3. Arquitectura Par a Par (P2P)	7
7. Transparencias en SGBDD	8
8. Homogeneidad versus Heterogeneidad	8
9. Fragmentacion de Datos	9
9.1. Fragmentacion Horizontal	9
9.2. Fragmentacion Vertical	10
9.3. Fragmentacion Mixta o Hibrida	10
9.4. Reglas de Correctitud de la Fragmentacion	10
10.Replicacion de Datos	11
10.1. Replicacion Sincrona versus Asincrona	11
10.2. Modelos de Escritura en Sistemas Replicados	11
10.3. Quorum y Consistencia $W+R>N$	12
11.Catalogo Distribuido	12
11.1. Tipos de Catalogo segun su Distribucion	12
11.2. Implementaciones Industriales del Catalogo	13
12.Procesamiento de Consultas Distribuidas	13

13.Transacciones Distribuidas y Control de Concurrencia	13
13.1. Protocolo de Bloqueo en Dos Fases (2PL)	14
13.2. Protocolo de Commit en Dos Fases (2PC)	14
13.3. Protocolo de Commit en Tres Fases (3PC)	14
13.4. Deadlock Distribuido	15
13.5. Control de Concurrencia Multiversion (MVCC)	15
14.Teorema CAP y Modelo PACELC en BDD	15
14.1. Modelo PACELC	16
15.Bases de Datos NoSQL en Entornos Distribuidos	16
15.1. Bases de Datos Documentales	16
15.2. Bases de Datos Clave-Valor	17
15.3. Bases de Datos Columnares (Wide Column)	17
15.4. Bases de Datos de Grafos	17
16.Ejemplos de Bases de Datos Distribuidas Reales	17
16.1. Google Spanner	17
16.2. CockroachDB	17
16.3. Apache Cassandra	18
16.4. Amazon DynamoDB	18
16.5. MongoDB	18
16.6. PostgreSQL Distribuido	18
17.Recuperacion ante Fallos en BDD	19
18.Ejercicio Practico de Implementacion	20
18.1. Caso de Estudio: Banco del Austro	20
18.1.1. Analisis CAP	20
18.1.2. Diseno de Distribucion	20
18.1.3. Arquitectura Elegida	20
18.2. Paso 1: Levantamiento del Cluster PostgreSQL con Docker	20
18.3. Paso 2: Proyecto Spring Boot con Multiples DataSources	22
18.4. Paso 3: Catalogo Distribuido	24
18.5. Paso 4: Servicio de Consulta Distribuida	25
18.6. Paso 5: Transferencia Cross-Site con 2PC Manual	26
18.7. Paso 6: API REST	29
18.8. Paso 7: Clase Principal Spring Boot	30
18.9. Paso 8: Maven pom.xml	30
18.10 Paso 9: Ejecucion y Pruebas	31
18.11 Resultados Esperados	31
18.12 Ejercicio Reto: Extension a CockroachDB	32
19.Conclusiones	32
Referencias	32

1. Introduccion

El presente material constituye la guía académica integral de la Unidad 3 de la asignatura *Aplicaciones Distribuidas* (ISR-701), correspondiente al séptimo nivel de la Carrera de Ingeniería de Software de la Universidad Técnica Estatal de Quevedo. Su propósito es ofrecer al estudiante una visión completa, formal y aplicada de las Bases de Datos Distribuidas (BDD), desde sus fundamentos teóricos hasta su implementación práctica con tecnologías contemporáneas.

El crecimiento exponencial del volumen de datos, la dispersión geográfica de las organizaciones y la demanda creciente de disponibilidad continua han convertido a las BDD en una tecnología central para los sistemas de información modernos. Comprender sus principios resulta indispensable para todo ingeniero de software que aspire a diseñar arquitecturas escalables, resilientes y consistentes.

El documento está organizado en tres ejes que reflejan las tres semanas del cronograma de la unidad. El primero introduce los conceptos fundamentales: definición formal, características, arquitecturas, transparencias y posicionamiento CAP. El segundo desarrolla el diseño físico de la distribución: fragmentación horizontal, vertical y mixta, estrategias de replicación y catálogo distribuido. El tercero aborda el control de concurrencia distribuido, los protocolos de commit, la detección de deadlocks y los modelos NoSQL como alternativa a los SGBDD relacionales.

Cada concepto se acompaña de definiciones formales, ejemplos de sistemas reales en producción (Google Spanner, CockroachDB, Cassandra, DynamoDB, MongoDB) y un ejercicio práctico completo de implementación sobre el caso de estudio *Banco del Austro*, utilizando Java 21, Spring Boot 3.x, PostgreSQL 16 y Docker. Todo el código es compilable y reproducible.

2. Definicion Formal de Base de Datos Distribuida

Definicion: Base de Datos Distribuida (BDD)

Una **base de datos distribuida** es una colección de bases de datos múltiples, lógicamente interrelacionadas, distribuidas sobre una red de computadores. Un **Sistema de Gestión de Bases de Datos Distribuidas** (SGBDD o DDBMS, por sus siglas en inglés) es el sistema de software que permite la gestión de la base de datos distribuida y hace que la distribución sea transparente para los usuarios [1].

La definición encierra dos requisitos esenciales que la distinguen de otros sistemas. Primero, la **interrelación lógica**: los datos almacenados en los distintos sitios no son meramente coexistentes, sino que conforman un esquema global coherente con relaciones entre tablas, integridad referencial y semántica unificada. Segundo, la **transparencia de la distribución**: la complejidad de la ubicación física, la fragmentación y la replicación queda oculta al usuario final y a las aplicaciones cliente.

Es fundamental diferenciar la BDD de tres conceptos cercanos pero distintos. Una **base de datos centralizada con acceso remoto** ofrece un único sitio físico que recibe consultas desde la red, pero no hay distribución de datos. Un **conjunto de bases de datos independientes** en distintos servidores carece de esquema lógico unificado y por tanto no satisface el primer requisito. Una **base de datos paralela** ejecuta consultas con múltiples procesadores sobre un mismo sitio físico para acelerar el procesamiento, sin distribución geográfica.

Diferencia esencial

La diferencia esencial entre una base de datos distribuida y una centralizada no radica en la tecnología de almacenamiento, sino en la **distribución geográfica o lógica de los datos** y en cómo el sistema gestiona la **transparencia** de esa distribución. Un cluster PostgreSQL con replicación síncrona en dos centros de datos constituye una BDD; dos bases PostgreSQL independientes que comparten datos por exportación CSV no.

3. Conceptos Relacionados con Bases de Datos Distribuidas

Antes de profundizar en la arquitectura y el diseño de las BDD conviene fijar el vocabulario técnico empleado a lo largo del documento. Estos términos aparecen tanto en la literatura académica como en la documentación industrial de los principales sistemas de producción.

3.1 Sitio o Nodo

Un **sitio** (también llamado nodo o site) es una unidad lógica de almacenamiento y procesamiento dentro de la BDD. Típicamente corresponde a un servidor físico con su propio motor SGBD local, su almacenamiento persistente y su porción del esquema. En despliegues virtualizados un sitio puede ser un contenedor Docker, una máquina virtual o un pod de Kubernetes. La conexión entre sitios se realiza por una red TCP/IP que puede ser una LAN del centro de datos, una WAN entre ciudades o el internet público.

3.2 Esquema Global y Esquemas Locales

El **esquema global** (o conceptual global) describe la base de datos completa como si fuera una sola entidad lógica. Define las relaciones (tablas), atributos, restricciones de integridad y reglas de negocio. El **esquema local** de cada sitio describe los fragmentos o réplicas que ese sitio almacena físicamente. La aplicación siempre conversa contra el esquema global; el SGBDD traduce las operaciones al esquema local apropiado.

3.3 Fragmento

Un **fragmento** es una porción de una relación del esquema global asignada a un sitio específico. Existen tres tipos: **horizontal** (subconjunto de filas), **vertical** (subconjunto de columnas) y **mixto** (combinación de los dos). El conjunto de todos los fragmentos de una relación debe permitir reconstruirla sin pérdida.

3.4 Réplica

Una **réplica** es una copia idéntica de un fragmento o relación completa que reside en más de un sitio. La replicación incrementa la disponibilidad y reduce la latencia de lectura, a costa de mayor complejidad en el mantenimiento de la consistencia entre copias.

3.5 Catálogo Distribuido

El **catálogo distribuido** (también llamado diccionario de datos distribuido o catálogo global) es la estructura de metadatos que indica donde reside cada fragmento y réplica. Es la pieza clave que habilita las transparencias de fragmentación y ubicación: cuando una consulta llega al SGBDD, este consulta primero el catálogo para determinar a qué sitios enrutarla.

3.6 Transacción Distribuida

Una **transacción distribuida** es una unidad lógica de trabajo que accede a datos en múltiples sitios y debe satisfacer las propiedades ACID (atomicidad, consistencia, aislamiento y durabilidad) de forma global. Su correcta ejecución exige protocolos de coordinación como el commit en dos fases (2PC).

3.7 Procesamiento de Consultas Distribuido

El **procesamiento de consultas distribuido** es el conjunto de técnicas que el SGBDD aplica para transformar una consulta global (escrita contra el esquema global) en un plan ejecutable que combina operaciones locales en cada sitio más operaciones de transferencia de datos por la red. El

optimizador distribuido busca minimizar el costo total, que ahora incluye no solo I/O y CPU sino tambien comunicacion.

3.8 Modelo de Consistencia

El **modelo de consistencia** es la garantia formal que el sistema ofrece sobre el orden y la visibilidad de las operaciones concurrentes. Va desde el modelo mas fuerte, la *serializabilidad estricta*, hasta los mas debiles, como la *consistencia eventual*. Cada modelo establece un balance distinto entre rendimiento, disponibilidad y simplicidad de programacion.

3.9 Tolerancia a Particiones de Red

Una **particion de red** ocurre cuando una falla de la red impide la comunicacion entre subconjuntos de sitios, aunque cada subconjunto siga funcionando internamente. La **tolerancia a particiones** es la capacidad del sistema para continuar operando bajo estas condiciones. El teorema CAP establece que ante una particion el sistema debe elegir entre preservar la consistencia o la disponibilidad.

4. Caracteristicas de las Bases de Datos Distribuidas

Las BDD presentan caracteristicas diferenciales que las hacen apropiadas para escenarios donde un sistema centralizado resultaria insuficiente. Estas caracteristicas constituyen tanto sus ventajas como las fuentes de su complejidad inherente.

4.1 Distribucion Fisica y Logica

Los datos residen en sitios geograficamente o logicamente separados, conectados por una red. La separacion puede ser intraciudad (varios centros de datos de la misma ciudad), interurbana (oficinas en distintas ciudades) o intercontinental (regiones cloud globales). En todos los casos el sistema mantiene la ilusion de un unico almacen logico.

4.2 Autonomia Local

Cada sitio puede operar de manera independiente cuando la red falla o cuando el resto del sistema esta indisponible. Las consultas y transacciones locales no se ven afectadas por fallos externos. Esta caracteristica es fundamental para la *disponibilidad continua* y para el cumplimiento de la regla 1 de Date.

4.3 Crecimiento Incremental

Anadir nuevos sitios a la BDD debe ser posible sin reescribir aplicaciones ni reorganizar masivamente los datos existentes. La escalabilidad horizontal es uno de los principales motivadores para migrar desde un SGBD centralizado.

4.4 Mayor Disponibilidad

La replicacion permite que la caida de un sitio no implique la perdida de servicio: las consultas se redirigen automaticamente a replicas activas. Sistemas con replicacion 3x y consenso por mayoria (quorum Raft o Paxos) toleran la perdida de un nodo sin interrupcion.

4.5 Mejor Rendimiento Localizado

Los datos pueden colocarse cerca de los usuarios que los consultan con mas frecuencia, reduciendo la latencia de red. Este principio, conocido como *localidad de referencia*, es la base del diseno de fragmentacion: ubicar las tuplas de Cuenca en el servidor de Cuenca, y las de Quito en el de Quito.

4.6 Heterogeneidad Potencial

Distintos sitios pueden ejecutar distintos motores SGBD (PostgreSQL en un sitio, Oracle en otro), siempre que exista una capa de federacion o middleware que unifique el acceso. La heterogeneidad permite integrar sistemas legados sin migrarlos.

4.7 Compromisos del Teorema CAP

Cualquier BDD debe declarar explicitamente su posicion en el teorema CAP: ante una particion de red, sacrifica consistencia (sistemas AP, como Cassandra) o disponibilidad (sistemas CP, como MongoDB con writeConcern majority). No existe el sistema ideal CA bajo particiones reales [3].

Trade-offs inherentes

Las características anteriores no son gratuitas. La distribucion introduce **complejidad de coordinacion**, **latencia de red en operaciones cross-site**, **dificultad en el mantenimiento de la integridad referencial**, y **problemas de consistencia** que no existen en un SGBD centralizado. Diseñar una BDD es ejercer un balance permanente entre estos compromisos.

5. Las 12 Reglas de Date para SGBDD

En 1987 C. J. Date formulo doce propiedades que deberia cumplir un SGBDD ideal. Aunque ninguna implementacion real las satisface al cien por ciento, constituyen el estandar de referencia academico e industrial para evaluar cualquier sistema distribuido de datos.

- R1. Autonomia local:** cada sitio gestiona su propia seguridad, integridad y operaciones sin depender de otros.
- R2. No dependencia de un sitio central:** no debe existir un nodo cuya caida paralice el sistema completo.
- R3. Operacion continua:** el sistema sigue disponible aunque fallen nodos individuales o se realicen tareas de mantenimiento.
- R4. Independencia de ubicacion:** el usuario no necesita conocer en que sitio reside un dato para acceder a el.
- R5. Independencia de fragmentacion:** las consultas se escriben contra el esquema global sin preocuparse por como esta dividida cada relacion.
- R6. Independencia de replicacion:** las aplicaciones operan como si existiera una unica copia de cada dato, aunque internamente haya varias.
- R7. Procesamiento de consultas distribuido:** el optimizador trata las consultas multi-sitio y elige el plan mas eficiente.
- R8. Manejo de transacciones distribuidas:** las propiedades ACID se garantizan globalmente mediante protocolos como 2PC.
- R9. Independencia de hardware:** el sistema funciona sobre maquinas con distinta arquitectura.
- R10. Independencia de sistema operativo:** los sitios pueden correr distintos SO sin afectar el funcionamiento global.
- R11. Independencia de red:** el protocolo de comunicacion subyacente (TCP/IP, RPC, gRPC) es transparente al usuario.

R12. Independencia de SGBD: distintos sitios pueden usar motores distintos, siempre que se cumpla un estandar comun (SQL, ODBC, JDBC).

Tabla 1: Grado aproximado de cumplimiento de las reglas de Date en sistemas reales

Regla	PostgreSQL FDW	CockroachDB	Cassandra	MongoDB Sharded
R1 Autonomia local	Alta	Media	Alta	Media
R2 Sin sitio central	Parcial	Total	Total	Total
R3 Operacion continua	Parcial	Total	Total	Total
R4 Indep. ubicacion	Total	Total	Total	Total
R5 Indep. fragmentacion	Parcial	Total	Total	Total
R6 Indep. replicacion	Parcial	Total	Total	Total
R7 Consultas distribuidas	Parcial	Total	Limitada	Parcial
R8 Trans. distribuidas	Parcial (2PC)	Total (Raft)	No ACID	Parcial
R9–R11 Independencias HW/SO/Red	Total	Total	Total	Total
R12 Indep. de SGBD	Total (FDW)	No	No	No

6. Arquitecturas de SGBDD

La literatura clasica identifica tres modelos arquitectonicos principales que se diferencian por el grado de autonomia entre los sitios participantes y por la presencia o ausencia de un coordinador central [1, 2].

6.1 Arquitectura Cliente-Servidor

En la arquitectura cliente-servidor un servidor centralizado gestiona la BDD y los clientes envian solicitudes a traves de la red. Aunque historicamente se la considera el modelo mas simple, en su variante distribuida moderna el "servidor" puede ser un cluster con replicacion primaria-secundaria. PostgreSQL con streaming replication o MySQL con replicacion asincrona son ejemplos tipicos.

La ventaja principal es la simplicidad operativa y la facilidad de gestion. La desventaja critica es que el servidor primario constituye un cuello de botella y un punto unico de fallo, mitigado pero no eliminado por la replicacion.

6.2 Arquitectura Federada

La arquitectura federada (tambien llamada multi-database o multi-DBMS) interconecta multiples SGBD locales preexistentes mediante una capa de federacion que ofrece una vista global unificada. Cada sitio conserva su autonomia y su esquema local; la capa intermedia traduce consultas globales en sub-consultas locales. PostgreSQL con Foreign Data Wrappers (FDW), Oracle Federated Database y Denodo son representantes de este modelo.

La ventaja principal es la capacidad de integrar sistemas heterogeneos preexistentes sin migrarlos. La desventaja es la complejidad de mantener la coherencia semantica entre esquemas locales y la dificultad de optimizar consultas que cruzan sitios.

6.3 Arquitectura Par a Par (P2P)

En la arquitectura par a par todos los nodos tienen el mismo rol: pueden recibir consultas, almacenar datos y participar en el consenso. No existe coordinador central. Apache Cassandra, ScyllaDB, Riak y, en cierto sentido, CockroachDB son ejemplos de este modelo. La comunicacion entre nodos emplea protocolos gossip para diseminar el estado del cluster.

La ventaja es la escalabilidad horizontal practicamente ilimitada y la maxima tolerancia a fallos. La desventaja es la complejidad de coordinacion: el consenso distribuido (Raft, Paxos) o las estructuras de hash consistente son indispensables.

Tabla 2: Comparativa de las tres arquitecturas SGBDD

Característica	Cliente-Servidor	Federada	P2P
Autonomia local	Baja	Alta	Alta
Escalabilidad horizontal	Limitada	Media	Muy alta
Complejidad operativa	Baja	Alta	Muy alta
Punto unico de fallo	Si	Parcial	No
Tolera heterogeneidad SGBD	No	Si	No
Ejemplo representativo	PostgreSQL replica	Oracle Federated	Apache Cassandra

7. Transparencias en SGBDD

El concepto de **transparencia** es el principio rector del diseno de SGBDD: ocultar al usuario y a la aplicacion la complejidad de la distribucion para que el sistema se perciba como si fuera centralizado. Se distinguen ocho tipos de transparencia que extienden las cuatro clasicas de los sistemas distribuidos generales.

Tabla 3: Las ocho transparencias del SGBDD

Tipo	Descripcion	Ejemplo
De red (acceso)	El usuario no conoce ni la topologia ni los protocolos de red	<code>SELECT * FROM clientes</code> funciona igual sin importar la red
De ubicacion	El usuario no conoce el sitio fisico donde residen los datos	La consulta no incluye la direccion IP del servidor
De fragmentacion	El usuario no sabe como se dividio cada relacion	Una tabla PEDIDOS puede estar fragmentada en cuatro nodos
De replicacion	El usuario no sabe cuantas copias del dato existen	Se lee un dato sin saber de cual replica proviene
De concurrencia	Las transacciones concurrentes se ven como si fueran seriales	No se observan estados intermedios de otras transacciones
De fallos	Los fallos de nodos individuales se manejan internamente	Si nodo A cae, la consulta se redirige al nodo B sin error visible
De rendimiento	El optimizador elige el plan mas eficiente automaticamente	Una consulta se redirige al nodo con menos carga
De migracion	Los datos pueden moverse entre sitios sin afectar las aplicaciones	Un rebalanceo de shards no requiere cambios de codigo cliente

Estas ocho transparencias coinciden parcialmente con las ocho transparencias ANSA estudiadas en la Unidad 1 (acceso, ubicacion, migracion, reubicacion, replicacion, concurrencia, fallos y persistencia), adaptadas al contexto especifico de los datos. La transparencia de *fragmentacion* es propia de las BDD y no aparece en sistemas distribuidos genericos.

8. Homogeneidad versus Heterogeneidad

Los SGBDD se clasifican en dos grandes familias segun la uniformidad de sus sitios.

Una BDD **homogenea** tiene en todos sus sitios el mismo motor SGBD, el mismo esquema, y la misma version del software. Esto simplifica la coordinacion, las transacciones distribuidas y el optimizador de consultas. Todos los sitios "hablan el mismo idioma" comparten convenciones de tipos, codificacion y semantica. CockroachDB, Cassandra y MongoDB Sharded Cluster son ejemplos puramente homogeneos.

Una BDD **heterogenea** integra sitios con motores SGBD distintos (por ejemplo, un sitio con PostgreSQL, otro con Oracle, otro con MySQL). Requiere una capa de federacion o middleware que traduzca consultas y reconcilie diferencias de tipos, semantica y modelo de datos. PostgreSQL con

FDW para conectarse a Oracle, o sistemas de virtualizacion de datos como Denodo, son ejemplos de este modelo.

Tabla 4: Comparativa homogenea vs heterogenea

Aspecto	Homogenea	Heterogenea
Esfuerzo de integracion	Bajo	Alto
Optimizacion de consultas distribuidas	Eficiente	Limitada
Transacciones distribuidas (2PC)	Soporte nativo	Requiere coordinador externo
Integracion con sistemas legados	Dificil	Facil
Coordinacion del catalogo	Sencilla	Requiere mapeos de esquema
Casos de uso tipicos	Nuevos despliegues	Modernizacion empresarial

9. Fragmentacion de Datos

Definicion: Fragmentacion

La **fragmentacion** es el proceso de dividir una relacion (tabla) en porciones llamadas **fragmentos**, que se almacenan en diferentes sitios de la red. Cada fragmento es subconjunto de la relacion original, y la union de todos los fragmentos debe reconstruir la relacion completa sin perdida ni duplicacion (excepto en esquemas hibridos con replicacion).

La fragmentacion busca el principio de **localidad de referencia**: los datos se almacenan donde se usan con mayor frecuencia, reduciendo el trafico de red y la latencia de acceso. Existen tres tipos formales: horizontal, vertical y mixta.

9.1 Fragmentacion Horizontal

La fragmentacion horizontal divide las **filas** (tuplas) de una tabla entre los sitios. Cada fragmento contiene un subconjunto de tuplas que satisfacen un predicado de seleccion. Formalmente se expresa con el operador de seleccion del algebra relacional:

$$F_i = \sigma_{p_i}(R), \quad i = 1, 2, \dots, n \quad (1)$$

donde R es la relacion original, p_i es el predicado que define al fragmento F_i , y los predicados son *minterminos* disjuntos que cubren todo el dominio de la clave de fragmentacion.

La reconstruccion de la relacion original se obtiene mediante la union:

$$R = F_1 \cup F_2 \cup \dots \cup F_n \quad (2)$$

Ejemplo: Fragmentacion horizontal de PEDIDOS por ciudad

Sea la relacion PEDIDOS(id, cliente, ciudad, monto). Definimos dos fragmentos:

$$F_1 = \sigma_{ciudad='Cuenca'}(PEDIDOS) \rightarrow \text{nodo Cuenca} \quad (3)$$

$$F_2 = \sigma_{ciudad='Quito'}(PEDIDOS) \rightarrow \text{nodo Quito} \quad (4)$$

Los pedidos de clientes de Cuenca se almacenan en el nodo Cuenca y los de Quito en el nodo Quito. La reconstruccion es $PEDIDOS = F_1 \cup F_2$.

Existe una variante llamada **fragmentacion horizontal derivada**, en la cual los fragmentos de una relacion se definen segun los de otra relacion relacionada. Por ejemplo, si LINEAS_PEDIDO tiene una clave foranea hacia PEDIDOS, sus filas pueden fragmentarse *junto con* las filas correspondientes de PEDIDOS, manteniendo la localidad de los JOIN.

9.2 Fragmentacion Vertical

La fragmentacion vertical divide las **columnas** (atributos) de una tabla. Cada fragmento incluye la **clave primaria** (para permitir la reconstruccion mediante JOIN) y un subconjunto de los demas atributos. Formalmente se expresa con el operador de proyeccion:

$$FV_i = \pi_{PK, A_i}(R), \quad i = 1, 2, \dots, n \quad (5)$$

donde PK es el conjunto de atributos clave primaria y A_i son subconjuntos disjuntos de atributos no clave. La reconstruccion se realiza con joins naturales:

$$R = FV_1 \bowtie FV_2 \bowtie \dots \bowtie FV_n \quad (6)$$

Ejemplo: Fragmentacion vertical de EMPLEADOS por sensibilidad

Sea EMPLEADOS(id, nombre, cargo, salario, depart, telefono). Separamos los datos organizacionales de los sensibles:

$$FV_1 = \pi_{id, nombre, cargo, depart}(EMPLEADOS) \rightarrow \text{nodo RRHH} \quad (7)$$

$$FV_2 = \pi_{id, salario, telefono}(EMPLEADOS) \rightarrow \text{nodo Nomina (cifrado)} \quad (8)$$

La reconstruccion es $EMPLEADOS = FV_1 \bowtie_{id} FV_2$. El atributo id aparece en ambos fragmentos por ser la clave primaria.

El diseno de fragmentacion vertical optimo utiliza **analisis de afinidad de atributos**: se construye una matriz que mide cuantas veces dos atributos son consultados juntos, y un algoritmo aglomerativo (Hoffer-Severance) agrupa los atributos con alta afinidad en el mismo fragmento.

9.3 Fragmentacion Mixta o Hibrida

La fragmentacion mixta combina horizontal y vertical. Se puede aplicar primero una fragmentacion horizontal y luego, sobre cada fragmento horizontal, una fragmentacion vertical (o viceversa). El resultado es un arbol de fragmentos del cual la reconstruccion requiere primero JOINS para deshacer la fragmentacion vertical y luego UNIONS para deshacer la horizontal.

Ejemplo: Fragmentacion mixta de VENTAS

La relacion VENTAS(id, vendedor, region, producto, monto) se fragmenta primero por region (Sierra y Costa) y luego el fragmento Sierra se fragmenta verticalmente para separar datos comerciales (vendedor) de datos contables (producto, monto). El fragmento Costa permanece sin fragmentar verticalmente.

9.4 Reglas de Correctitud de la Fragmentacion

Todo esquema de fragmentacion debe satisfacer tres reglas formales para garantizar que la BDD se comporte como una unica base de datos logica:

- C1. Completitud:** cada dato de la relacion original debe pertenecer a, al menos, un fragmento. Formalmente, $\bigcup_i F_i = R$. La violacion tipica es omitir una fila o columna en todos los fragmentos.
- C2. Reconstruccion:** la relacion original debe poder reconstruirse a partir de los fragmentos mediante un operador del algebra relacional (UNION para horizontal, JOIN para vertical). La violacion tipica es omitir la clave primaria en algun fragmento vertical, lo que impide el JOIN.

C3. Disjuncion: para la fragmentacion horizontal, ningun dato debe aparecer en mas de un fragmento (excepto la clave en la vertical). $F_i \cap F_j = \emptyset$ para $i \neq j$. La violacion tipica es predicados solapados que asignan la misma fila a dos sitios, generando inconsistencias.

Tabla 5: Reglas de correctitud por tipo de fragmentacion

Regla	Horizontal	Vertical	Mixta
Compleitud	Si	Si	Si
Reconstruccion via UNION	Si	No	Parcial
Reconstruccion via JOIN	No	Si	Parcial
Disjuncion en filas	Si	No aplica	Parcial
Clave primaria en cada fragmento	No requerida	Obligatoria	Obligatoria

10. Replicacion de Datos

Definicion: Replicacion

La **replicacion** es el proceso de mantener copias identicas de los datos (relaciones, fragmentos o tuplas individuales) en multiples sitios. A diferencia de la fragmentacion –donde un dato reside en un unico sitio–, en la replicacion el mismo dato existe en varios sitios. El objetivo es mejorar la **disponibilidad** y la **latencia de lectura**, a costa de mayor complejidad para mantener la **consistencia**.

10.1 Replicacion Sincrona versus Asincrona

La replicacion **sincrona** confirma una escritura al cliente solo despues de que todas las replicas (o un quorum de ellas) hayan confirmado haber recibido y persistido el cambio. Garantiza consistencia fuerte: todas las copias estan siempre actualizadas. El costo es la latencia: la transaccion espera la confirmacion mas lenta. PostgreSQL con `synchronous_commit = on` y `synchronous_standby_names` configurado es un ejemplo.

La replicacion **asincrona** confirma la escritura al cliente inmediatamente despues de persistir en la replica primaria, y propaga los cambios al resto en segundo plano. Ofrece menor latencia, pero introduce un **lag de replicacion** durante el cual las replicas pueden devolver datos obsoletos. Si el primario falla antes de propagar, los ultimos cambios pueden perderse. MySQL con replicacion estandar y MongoDB con `writeConcern: 1` son ejemplos.

Tabla 6: Replicacion sincrona vs asincrona

Aspecto	Sincrona	Asincrona
Consistencia	Fuerte	Eventual
Latencia de escritura	Alta	Baja
Disponibilidad ante falla de replica	Reducida	Alta
Riesgo de perdida de datos	Ninguno	Si (lag no propagado)
Caso de uso tipico	Banca, inventario critico	Redes sociales, logs
Ejemplo tecnologia	PostgreSQL synchronous standby	MySQL async replica

10.2 Modelos de Escritura en Sistemas Replicados

Existen tres modelos principales para gestionar las escrituras en un sistema con replicas [2].

El modelo **primario-secundario** (single-leader) designa una replica como primaria, unica autorizada para aceptar escrituras. Las secundarias reciben las escrituras propagadas y solo sirven

lecturas. Es el modelo mas simple y soporta consistencia fuerte. Su desventaja es que el primario es un cuello de botella de escritura y un punto unico de fallo (mitigado por failover automatico).

El modelo **multi-primario** (multi-leader) permite a varias replicas aceptar escrituras concurrentes. Mejora la disponibilidad de escritura y reduce la latencia para usuarios geograficamente distribuidos, pero introduce **conflictos de escritura** que deben resolverse con politicas (last-write-wins, vector clocks, CRDTs). MySQL Group Replication, Galera Cluster y Couchbase XDCR usan este modelo.

El modelo **sin lider** (leaderless) elimina la distincion entre primario y replica. El cliente escribe en varios nodos en paralelo y considera la escritura exitosa si una cantidad minima (W) confirma. Para lecturas, el cliente consulta varios nodos (R) y compara versiones. La condicion de consistencia es $W + R > N$, donde N es el numero total de replicas. Apache Cassandra, Riak y Amazon DynamoDB siguen este modelo.

10.3 Quorum y Consistencia $W+R>N$

En sistemas sin lider, el quorum proporciona consistencia ajustable. Con N replicas totales, el cliente puede elegir cuantas replicas deben confirmar una escritura (W) y cuantas debe leer (R). La condicion $W + R > N$ garantiza que cualquier lectura intersecte con la ultima escritura, asegurando que se obtenga al menos un valor actualizado.

Ejemplo: Configuracion de quorum tipica

Con $N = 5$ replicas, una configuracion comun es $W = 3$ y $R = 3$, ya que $3 + 3 = 6 > 5$. Esto tolera la falla de hasta dos nodos sin perder disponibilidad de escritura ni de lectura, manteniendo consistencia. Si se prefiere optimizar lectura, se puede usar $W = 4$, $R = 2$, tolerando una falla de lectura pero exigiendo cuatro confirmaciones en escritura.

11. Catalogo Distribuido

Definicion: Catalogo Distribuido

El **catalogo distribuido** (tambien diccionario de datos distribuido o catalogo global) es la estructura de metadatos que describe **donde residen** los fragmentos y replicas de cada relacion del esquema global. Cuando una consulta llega al SGBDD, este consulta primero el catalogo para determinar a que sitios enrutar las sub-consultas. Es la pieza clave que habilita las transparencias de fragmentacion y ubicacion.

11.1 Tipos de Catalogo segun su Distribucion

El propio catalogo, al ser un conjunto de datos critico, debe distribuirse cuidadosamente. Se distinguen tres estrategias.

El **catalogo centralizado** reside en un unico nodo (frecuentemente el coordinador). Es simple de implementar y mantener consistente, pero introduce un punto unico de fallo: si el nodo del catalogo cae, ninguna consulta global puede enrutarse. Es la estrategia menos resiliente y solo se justifica en despliegues pequenos.

El **catalogo completamente replicado** mantiene una copia identica en cada nodo. Cualquier consulta puede resolver el enrutamiento localmente sin red, lo que minimiza latencia. La desventaja es el costo de mantener todas las copias sincronizadas cuando se anaden o quitan fragmentos: cada cambio requiere una transaccion distribuida sobre el catalogo.

El **catalogo parcialmente replicado** (o dividido) reparte la informacion: cada nodo guarda los metadatos de los fragmentos locales mas la informacion de los nodos con los que interactua

frecuentemente. Ofrece un balance entre disponibilidad y costo de actualizacion. Es la estrategia adoptada por Apache Cassandra y CockroachDB.

11.2 Implementaciones Industriales del Catalogo

En sistemas modernos el catalogo distribuido se implementa con tecnologias de coordinacion especializadas. **Apache ZooKeeper**, **etcd** y **HashiCorp Consul** son almacenes de clave-valor consistentes basados en consenso (Zab o Raft) usados como catalogo en clusters de Hadoop, Kubernetes y CockroachDB. Almacenan informacion sobre nodos vivos, particiones, lideres y metadatos de esquema, garantizando que todos los participantes vean la misma vista del cluster.

12. Procesamiento de Consultas Distribuidas

El procesamiento de una consulta SQL en una BDD pasa por cuatro fases consecutivas, gestionadas por el optimizador distribuido.

La **descomposicion** traduce la consulta SQL del usuario a una expresion del algebra relacional sobre el esquema global, valida sintaxis, semantica y resuelve referencias a vistas.

La **localizacion** convierte la consulta global en una consulta sobre fragmentos especificos, usando el catalogo. Por ejemplo, una consulta `SELECT * FROM PEDIDOS WHERE ciudad = 'Cuenca'` se redirige al fragmento F_1 del nodo Cuenca, evitando consultar los demas.

La **optimizacion global** elige el plan de ejecucion mas eficiente considerando los costos de I/O, CPU y, sobre todo, comunicacion entre sitios. Las decisiones tipicas incluyen donde realizar los JOINS (semi-joins, bloom-joins), si se replica un fragmento temporalmente para reducir red, y en que orden se materializan los resultados intermedios.

La **ejecucion** despacha las sub-consultas a los nodos correspondientes, recibe los resultados parciales y los combina segun el plan. El coordinador puede ser el nodo que recibio la consulta o un nodo elegido dinamicamente segun la localizacion de los datos.

Tabla 7: Tecnicas de optimizacion en consultas distribuidas

Tecnica	Descripcion
Filtrado temprano	Aplicar selecciones y proyecciones antes de transferir datos
Semi-join	Transferir solo claves para reducir el tamano de los datos enviados en un join
Bloom-join	Usar filtros de Bloom para reducir el coste del semi-join
Replicacion temporal	Copiar un fragmento pequeno a varios nodos para evitar joins distribuidos
Particion-wise join	Realizar joins entre fragmentos co-particionados localmente
Caches de resultados	Reutilizar resultados intermedios entre consultas

13. Transacciones Distribuidas y Control de Concurrency

Definicion: Transaccion Distribuida

Una **transaccion distribuida** es una unidad logica de trabajo que accede a datos en multiples sitios. Debe cumplir las propiedades **ACID** de manera global: atomicidad (todos o ningun nodo aplica el cambio), consistencia (el estado final es valido), aislamiento (no se observan estados intermedios) y durabilidad (los cambios confirmados persisten ante fallos).

13.1 Protocolo de Bloqueo en Dos Fases (2PL)

El **Two-Phase Locking** (2PL) es el mecanismo clasico para garantizar la **serializabilidad** de transacciones concurrentes. Cada transaccion atraviesa dos fases:

Fase 1: Crecimiento: la transaccion solo puede **adquirir** bloqueos sobre los recursos que necesita; no puede liberar ninguno.

Fase 2: Decrecimiento: una vez que la transaccion libera el primer bloqueo, no puede adquirir mas. Solo puede liberar los que ya posee.

La variante **2PL Estricto** mantiene todos los bloqueos hasta el **COMMIT** o **ROLLBACK** de la transaccion, lo que evita problemas de *lecturas sucias* y simplifica la recuperacion. Es la implementacion adoptada por la mayoria de SGBD comerciales.

En entornos distribuidos el 2PL requiere un **gestor de bloqueos** que puede ser centralizado (un nodo unico arbitra todos los bloqueos) o distribuido (cada nodo gestiona sus propios bloqueos, con coordinacion para deteccion de deadlocks). Los bloqueos se clasifican segun su modo: **compartido** (S, para lectura), **exclusivo** (X, para escritura), **actualizacion** (U, previo a escritura) e **intencion** (IS, IX, para jerarquias).

13.2 Protocolo de Commit en Dos Fases (2PC)

Definicion: Two-Phase Commit (2PC)

El **commit en dos fases** es el protocolo estandar para garantizar la **atomicidad** de una transaccion distribuida. Coordina a varios participantes (los sitios que tocaron datos) para que todos confirmen o todos aborten la transaccion, evitando estados intermedios inconsistentes.

El protocolo procede en dos fases dirigidas por un **coordinador**:

Fase 1: Preparacion (Voting): el coordinador envia un mensaje **PREPARE** a todos los participantes. Cada participante intenta ejecutar la parte local de la transaccion, escribe un registro **PREPARED** en su log de escritura anticipada (WAL) y responde **VOTE-COMMIT** si pudo o **VOTE-ABORT** si fallo.

Fase 2: Decision (Commit/Abort): si todos respondieron **VOTE-COMMIT**, el coordinador escribe **COMMIT** en su log y envia **GLOBAL-COMMIT** a todos. Si alguno respondio **VOTE-ABORT** o no respondio en un timeout, envia **GLOBAL-ABORT**. Cada participante aplica la decision y libera bloqueos.

Limitacion fundamental del 2PC

El 2PC es un protocolo **bloqueante**: si el coordinador falla despues de la fase de preparacion pero antes de comunicar la decision, los participantes quedan en estado *prepared* reteniendo bloqueos, sin saber si confirmar o abortar. Pueden quedar bloqueados indefinidamente. Esta limitacion motivo la creacion del 3PC y, mas tarde, de los protocolos de consenso como Paxos y Raft, que toleran fallos del coordinador.

13.3 Protocolo de Commit en Tres Fases (3PC)

El **3PC** extiende el 2PC con una fase intermedia **pre-commit** para evitar el bloqueo ante fallo del coordinador. Tras la votacion exitosa el coordinador envia un **PRE-COMMIT** que reciben los participantes, y solo despues envia el **COMMIT** final. Si el coordinador falla durante el pre-commit, los participantes pueden elegir un nuevo coordinador que conozca el estado y decida **COMMIT** o **ABORT** sin ambigüedad.

En la practica el 3PC es raramente usado en produccion porque asume un *modelo de red sincrono* (mensajes con retardo acotado) que rara vez se cumple. Los sistemas modernos prefieren protocolos de consenso como **Raft** [5] o **Paxos**, que tambien resuelven el problema sin asumir sincronia.

13.4 Deadlock Distribuido

Un **deadlock** (interbloqueo) ocurre cuando dos o mas transacciones esperan indefinidamente mutuamente para liberar recursos. En un entorno distribuido el grafo de espera se extiende entre nodos, lo que dificulta su deteccion.

Las estrategias classicas son:

- **Prevencion por ordenamiento de recursos:** las transacciones adquieren bloqueos siempre en el mismo orden global (por ejemplo, por identificador alfabetico de recurso). Elimina deadlocks por construccion pero reduce el paralelismo.
- **Prevencion por timestamps (Wait-Die, Wound-Wait):** cada transaccion recibe un timestamp al iniciar; se aborta la mas joven (Wait-Die) o la mas vieja (Wound-Wait) cuando hay conflicto.
- **Deteccion por grafo de espera:** el sistema construye un grafo dirigido $T_i \rightarrow T_j$ si T_i espera un recurso que T_j tiene. Un ciclo en el grafo indica deadlock; el algoritmo elige una transaccion *victima* y la aborta.
- **Deteccion por timeout:** si una transaccion espera mas de un umbral, se asume deadlock y se aborta. Simple pero genera falsos positivos.

13.5 Control de Concurrency Multiversion (MVCC)

El **MVCC** es una alternativa al 2PL clasico que evita bloqueos de lectura. Cada escritura crea una nueva *version* del registro etiquetada con el identificador de la transaccion. Cada transaccion ve una *snapshot* consistente del estado del sistema al momento en que inicio. Las lecturas nunca bloquean a las escrituras ni viceversa.

PostgreSQL implementa MVCC desde sus origenes. Cada tupla almacena dos campos: **xmin** (identificador de la transaccion que la inserto) y **xmax** (identificador de la transaccion que la elimino o reemplazo). Una transaccion con identificador T ve una version si su $xmin \leq T$ y $xmax > T$ o $xmax = NULL$. Las versiones obsoletas se eliminan con el proceso **VACUUM**. CockroachDB y MongoDB usan variantes de MVCC sobre estructuras LSM-Tree.

14. Teorema CAP y Modelo PACELC en BDD

El **teorema CAP**, formulado por Eric Brewer en 2000 y formalizado por Gilbert y Lynch en 2002, establece que un sistema distribuido no puede garantizar simultaneamente las tres propiedades siguientes [3]:

- **C** (Consistency): toda lectura recibe el dato mas reciente o un error.
- **A** (Availability): toda peticion recibe una respuesta (no error), aunque pueda no ser la mas reciente.
- **P** (Partition tolerance): el sistema continua operando incluso cuando la red presenta particiones.

Como las particiones de red son inevitables en sistemas reales, el verdadero compromiso es entre **C** y **A**: ante una particion, el sistema debe elegir consistencia (CP) o disponibilidad (AP).

Los sistemas **CP** (MongoDB con `writeConcern majority`, HBase, ZooKeeper) prefieren rechazar peticiones antes que servir datos potencialmente obsoletos. Son apropiados para banca, inventario critico, sistemas de pago.

Los sistemas **AP** (Cassandra, DynamoDB, CouchDB) prefieren servir peticiones con datos posiblemente desactualizados antes que rechazarlas. Son apropiados para redes sociales, catalogos, logs.

14.1 Modelo PACELC

Daniel Abadi propuso en 2012 el modelo **PACELC** como extension del CAP: "*si hay Particion (P), elegir entre Availability y Consistency; Else (E), elegir entre Latency y Consistency*" [4]. El modelo reconoce que incluso sin particiones, todo sistema distribuido enfrenta el dilema entre minimizar latencia y maximizar consistencia.

Tabla 8: Clasificacion PACELC de sistemas conocidos

Sistema	Bajo particion (PAC)	Sin particion (ELC)
MongoDB (majority)	PC	EC
HBase	PC	EC
Google Spanner	PC	EC
CockroachDB	PC	EC
Cassandra	PA	EL
DynamoDB	PA	EL
Riak	PA	EL
PostgreSQL (async replica)	PA	EL

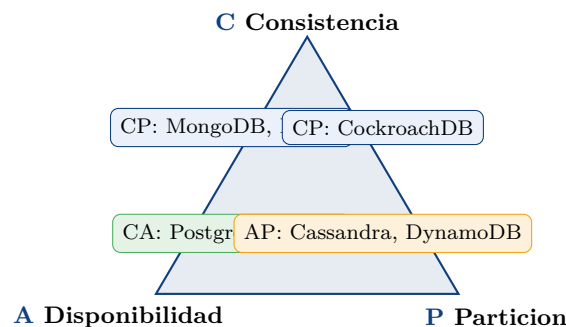


Figura 1: Posicionamiento de SGBDD conocidos en el triángulo CAP

15. Bases de Datos NoSQL en Entornos Distribuidos

Las bases de datos **NoSQL** (Not Only SQL) son sistemas que no usan el modelo relacional clasico. Estan diseñadas para escalabilidad horizontal nativa, esquemas flexibles y patrones de acceso especificos. Se agrupan en cuatro familias segun su modelo de datos.

15.1 Bases de Datos Documentales

Almacenan los datos como **documentos** (tipicamente JSON o BSON) agrupados en **colec-ciones**. Cada documento puede tener un esquema distinto. **MongoDB**, **CouchDB**, **Amazon DocumentDB** y **Azure Cosmos DB** (modo Mongo API) son representantes. Permiten anidacion natural de objetos y arrays, eliminando muchos JOINS tipicos del modelo relacional. Son apropiadas para catalogos de productos, perfiles de usuario y configuraciones flexibles.

15.2 Bases de Datos Clave-Valor

El modelo mas simple: una asociacion entre clave (string o binario) y valor (cualquier blob). **Redis**, **Memcached**, **Amazon DynamoDB** (en modo clave-valor) y **Riak** son ejemplos. Ofrecen latencias submilisegundo y enorme rendimiento, pero las operaciones se limitan a get/put/delete. Son la base de caches, sesiones, carritos de compra y colas ligeras.

15.3 Bases de Datos Columnares (Wide Column)

Organizan los datos por columnas en lugar de por filas. **Apache Cassandra**, **Apache HBase**, **Google Bigtable** y **ScyllaDB** son los principales. Cada fila puede tener un conjunto distinto de columnas; el almacenamiento por columnas optimiza consultas analiticas y compresion. Son apropiadas para series de tiempo, telemetria IoT, logs masivos y eventos.

15.4 Bases de Datos de Grafos

Modelan los datos como **nodos** y **aristas** con propiedades. **Neo4j**, **JanusGraph**, **Amazon Neptune** y **TigerGraph** son representantes. Optimizan consultas que recorren relaciones de varios saltos (amigos-de-amigos, deteccion de fraude, rutas en redes). Soportan lenguajes como Cypher (Neo4j) y Gremlin (JanusGraph).

Tabla 9: Las cuatro familias NoSQL y sus casos de uso

Familia	Ejemplos	Casos de uso tipicos
Documentales	MongoDB, CouchDB	Catalogos, perfiles, configuraciones
Clave-Valor	Redis, DynamoDB	Caches, sesiones, contadores
Columnares	Cassandra, HBase	Series de tiempo, IoT, logs
Grafos	Neo4j, JanusGraph	Redes sociales, fraude, rutas

16. Ejemplos de Bases de Datos Distribuidas Reales

A continuacion se presentan los seis sistemas mas relevantes del ecosistema actual, con sus caracteristicas tecnicas verificables y su posicionamiento PACELC.

16.1 Google Spanner

Google Spanner es la BDD globalmente distribuida desarrollada por Google. Combina la escalabilidad horizontal de los sistemas NoSQL con las garantias ACID y SQL del mundo relacional. Su innovacion clave es **TrueTime**, una API de tiempo con cota de incertidumbre garantizada gracias a relojes atomicos y receptores GPS en cada centro de datos. Spanner usa el protocolo **Paxos** para replicacion y un protocolo de commit distribuido con **2PC** sobre Paxos para transacciones cross-particion. Se posiciona como sistema PC/EC: ofrece consistencia externa (serializabilidad estricta) sacrificando latencia.

16.2 CockroachDB

CockroachDB es un SGBDD relacional de codigo abierto disenado para escalabilidad horizontal y resistencia a fallos. Compatible con el protocolo PostgreSQL en su capa SQL, internamente fragmenta automaticamente las tablas en **ranges** de 64 MB que se replican (3x o mas) con consenso **Raft**. Cada transaccion distribuida usa un protocolo de commit propietario inspirado en 2PC pero con optimizaciones de paralelismo. Garantiza serializabilidad estricta a nivel global. Su posicion PACELC es PC/EC.

16.3 Apache Cassandra

Apache Cassandra es una BDD NoSQL columnar diseñada para escalabilidad horizontal masiva y alta disponibilidad. Sigue el modelo P2P sin lider: todos los nodos son iguales y cualquiera puede aceptar escrituras. La replicacion se realiza con quorum configurable (N , W , R) y la consistencia es eventual por defecto, aunque puede subirse a "quorum.º .all". Se usa en cargas masivas: Netflix (recomendaciones), Apple (mensajería), Instagram (mensajes). Su posición es PA/EL.

16.4 Amazon DynamoDB

Amazon DynamoDB es el servicio gestionado de BDD clave-valor de AWS. Ofrece escalado automatico, latencias submilisegundo y replicacion multi-region opcional (Global Tables). Implementa consistencia eventual por defecto pero permite lecturas strongly consistent a doble precio. Usa hash consistente y sharding automatico transparente. Se posiciona como PA/EL.

16.5 MongoDB

MongoDB es la BDD documental de mayor adopcion. Implementa replicacion primaria-secundaria con failover automatico (Replica Sets) y sharding horizontal (Sharded Clusters) coordinado por procesos `mongos` y un catalogo en `config servers`. Desde la version 4.0 soporta transacciones ACID multi-documento. Su consistencia es ajustable mediante `writeConcern` y `readConcern`. Con `writeConcern: majority` se posiciona como PC/EC; con `writeConcern: 1` como PA/EL.

16.6 PostgreSQL Distribuido

PostgreSQL no es nativamente distribuido, pero soporta multiples patrones: **streaming replication** sincrona o asincrona para alta disponibilidad, **logical replication** para sincronizacion selectiva entre clusters, **Foreign Data Wrappers (FDW)** para acceso federado, y **Citus** (extension) para sharding horizontal. Es la base preferida para BDD heterogeneas y casos donde el ecosistema SQL maduro es prioritario.

Tabla 10: Comparativa tecnica de BDD reales

Sistema	Modelo	Consenso	PACELC	Replicacion	Uso tipi- co
Spanner	Relacional	Paxos+TrueTime	PC/EC	Multi-region	Banca glo- bal Goo- gle Ads
CockroachDB	Relacional	Raft	PC/EC	Multi-region	Banca, re- tail
Cassandra	Columnar	Gossip + Hinted Handoff	PA/EL	Configurable	IoT, logs, reco- men- da- cio- nes
DynamoDB	Clave-Valor	Quorum	PA/EL	Multi-AZ	Sesiones, ca- rri- tos AWS
MongoDB	Documental	Raft (oplog)	Configurable	Replica Set + Shards	Catalogos, per- files
PostgreSQL + Citus	Relacional	2PC + WAL	PC/EC	Streaming + sharding	ERP, ana- liti- ca

17. Recuperacion ante Fallos en BDD

La recuperacion en BDD extiende los mecanismos clasicos del SGBD centralizado (logs WAL, checkpoints, rollback) con consideraciones de coordinacion entre sitios.

Write-Ahead Logging (WAL): cada sitio mantiene un log local de todas las operaciones, persistido en disco antes de aplicar los cambios a la base. En caso de fallo, el log permite reconstruir el estado consistente al reiniciar (recuperacion ARIES).

Checkpoints: periodicamente cada sitio escribe un checkpoint que captura el estado de las transacciones activas, lo que reduce el tiempo de recuperacion al limitar cuanto log debe replicarse.

Logging del coordinador (2PC): el coordinador escribe el COMMIT o ABORT global en su log antes de comunicarlo. Si el coordinador falla, los participantes consultan el log de un nuevo coordinador para recuperar la decision.

Replicacion sincrona como mecanismo de recuperacion: si una replica falla, otra puede tomar su lugar (failover) sin perdida de datos, siempre que la replicacion sea sincrona o casi sincrona.

Snapshot y point-in-time recovery (PITR): muchos SGBDD permiten restaurar el sistema a un instante exacto del pasado combinando un snapshot completo con el log WAL posterior. Critico para corregir errores logicos (un DELETE accidental).

Consenso (Raft, Paxos) como base de recuperacion: en sistemas modernos el log replicado por consenso reemplaza al log local: cualquier nodo con el log puede reconstruir el estado. CockroachDB y etcd son ejemplos paradigmaticos.

18. Ejercicio Practico de Implementacion

Esta seccion desarrolla, paso a paso, un ejercicio completo de implementacion de una BDD homogenea federada con **PostgreSQL 16**, orquestada con **Docker Compose** y consumida desde una aplicacion **Spring Boot 3.x** en **Java 21**. El caso de estudio es el sistema de informacion del **Banco del Austro**, con sucursales en Cuenca (sede central) y Quito.

18.1 Caso de Estudio: Banco del Austro

El Banco del Austro requiere distribuir su base de datos con los siguientes requisitos no funcionales:

- Las consultas de saldo deben responder en menos de 200 milisegundos desde cualquier oficina.
- Las transferencias bancarias deben cumplir ACID estricto.
- La caida de una oficina no debe afectar la operacion de las demas.
- Los datos de clientes son compartidos entre oficinas (acceso multi-sucursal).
- Los datos de transacciones locales son propios de cada oficina.

18.1.1 Analisis CAP

Las transferencias requieren consistencia fuerte (ACID), por lo que el sistema se posiciona en **CP**. Ante una particion de red, preferimos bloquear la operacion en lugar de transferir dinero incorrectamente.

18.1.2 Diseno de Distribucion

Definimos el esquema de distribucion para tres tablas principales:

- Tabla **CLIENTES**: **replicada** en todos los nodos. Justificacion: lectura frecuente desde cualquier oficina, escritura infrecuente (alta de clientes).
- Tabla **CUENTAS**: **fragmentada horizontalmente** por oficina (F_{Cuenca} , F_{Quito}) con **replicacion** de fragmentos en la sede central. Justificacion: localidad para operaciones del titular en su oficina.
- Tabla **TRANSACCIONES**: **fragmentada horizontalmente** por oficina sin replicacion. Justificacion: log de operaciones locales.

18.1.3 Arquitectura Elegida

Arquitectura federada con coordinador en Cuenca. Cada oficina tiene un PostgreSQL local con autonomia para consultas locales. Las transferencias cross-site usan una transaccion coordinada con 2PC (en este ejercicio simulada en la capa de aplicacion).

18.2 Paso 1: Levantamiento del Cluster PostgreSQL con Docker

Creamos el archivo `docker-compose-bdd.yml` en la raiz del proyecto.

```
1 services:
2   pg-cuenca:
3     image: postgres:16-alpine
4     container_name: pg_cuenca
5     environment:
6       POSTGRES_DB: banco_cuenca
7       POSTGRES_USER: banco
8       POSTGRES_PASSWORD: banco123
```

```

9     ports:
10         - "5432:5432"
11     volumes:
12         - pg_cuenca_data:/var/lib/postgresql/data
13         - ./sql/init_cuenca.sql:/docker-entrypoint-initdb.d/init.sql
14     networks:
15         - banco_net
16     healthcheck:
17         test: ["CMD-SHELL", "pg_isready -U banco -d banco_cuenca"]
18         interval: 5s
19         retries: 5
20 pg-quito:
21     image: postgres:16-alpine
22     container_name: pg_quito
23     environment:
24         POSTGRES_DB: banco_quito
25         POSTGRES_USER: banco
26         POSTGRES_PASSWORD: banco123
27     ports:
28         - "5433:5432"
29     volumes:
30         - pg_quito_data:/var/lib/postgresql/data
31         - ./sql/init_quito.sql:/docker-entrypoint-initdb.d/init.sql
32     networks:
33         - banco_net
34     healthcheck:
35         test: ["CMD-SHELL", "pg_isready -U banco -d banco_quito"]
36         interval: 5s
37         retries: 5
38 volumes:
39     pg_cuenca_data:
40     pg_quito_data:
41     networks:
42     banco_net:
43         driver: bridge

```

Listing 1: docker-compose-bdd.yml: dos nodos PostgreSQL en contenedores Docker

Los scripts de inicializacion crean las tablas y datos de prueba en cada nodo. El nodo Cuenca contiene el fragmento F_{Cuenca} de CUENTAS y TRANSACCIONES.

```

1 CREATE TABLE IF NOT EXISTS clientes (
2     id          SERIAL PRIMARY KEY,
3     cedula      VARCHAR(10) UNIQUE NOT NULL,
4     nombre      VARCHAR(100) NOT NULL,
5     ciudad      VARCHAR(50) NOT NULL
6 );
7
8 CREATE TABLE IF NOT EXISTS cuentas (
9     id          SERIAL PRIMARY KEY,
10    numero      VARCHAR(20) UNIQUE NOT NULL,
11    cliente_id   INTEGER REFERENCES clientes(id),
12    saldo        NUMERIC(15,2) DEFAULT 0.00 CHECK (saldo >= 0),
13    oficina      VARCHAR(50) NOT NULL DEFAULT 'Cuenca'
14 );
15
16 CREATE TABLE IF NOT EXISTS transacciones (
17     id          SERIAL PRIMARY KEY,
18     cuenta_origen VARCHAR(20),
19     cuenta_destino VARCHAR(20),
20     monto        NUMERIC(15,2) NOT NULL,
21     fecha        TIMESTAMP DEFAULT NOW(),
22     estado        VARCHAR(20) DEFAULT 'COMPLETADA'
23 );
24

```

```

25 INSERT INTO clientes (cedula, nombre, ciudad) VALUES
26 ('0101234567', 'Maria Ordonez', 'Cuenca'),
27 ('0109876543', 'Carlos Vintimilla', 'Loja'),
28 ('0105551234', 'Ana Mosquera', 'Machala');
29
30 INSERT INTO cuentas (numero, cliente_id, saldo, oficina) VALUES
31 ('2201000001', 1, 5000.00, 'Cuenca'),
32 ('2201000002', 2, 1200.50, 'Cuenca'),
33 ('2201000003', 3, 3400.00, 'Cuenca');

```

Listing 2: sql/init_cuenca.sql: esquema y datos del nodo Cuenca

```

1 CREATE TABLE IF NOT EXISTS clientes (
2   id          SERIAL PRIMARY KEY,
3   cedula      VARCHAR(10) UNIQUE NOT NULL,
4   nombre      VARCHAR(100) NOT NULL,
5   ciudad      VARCHAR(50) NOT NULL
6 );
7
8 CREATE TABLE IF NOT EXISTS cuentas (
9   id          SERIAL PRIMARY KEY,
10  numero       VARCHAR(20) UNIQUE NOT NULL,
11  cliente_id   INTEGER REFERENCES clientes(id),
12  saldo        NUMERIC(15,2) DEFAULT 0.00 CHECK (saldo >= 0),
13  oficina      VARCHAR(50) NOT NULL DEFAULT 'Quito'
14 );
15
16 CREATE TABLE IF NOT EXISTS transacciones (
17   id          SERIAL PRIMARY KEY,
18   cuenta_origen VARCHAR(20),
19   cuenta_destino VARCHAR(20),
20   monto        NUMERIC(15,2) NOT NULL,
21   fecha        TIMESTAMP DEFAULT NOW(),
22   estado       VARCHAR(20) DEFAULT 'COMPLETADA'
23 );
24
25 INSERT INTO clientes (cedula, nombre, ciudad) VALUES
26 ('1701234567', 'Pedro Jimenez', 'Quito'),
27 ('1709876543', 'Luisa Paredes', 'Ibarra');
28
29 INSERT INTO cuentas (numero, cliente_id, saldo, oficina) VALUES
30 ('1701000001', 1, 8000.00, 'Quito'),
31 ('1701000002', 2, 2500.00, 'Quito');

```

Listing 3: sql/init_quito.sql: esquema y datos del nodo Quito

Levantamos el cluster con `docker compose -f docker-compose-bdd.yml up -d` y verificamos que ambos contenedores esten en estado `healthy` con `docker compose ps`.

18.3 Paso 2: Proyecto Spring Boot con Multiples DataSources

Creamos el proyecto Spring Boot mediante Spring Initializr con las dependencias: Spring Web, Spring Data JPA, PostgreSQL Driver y Lombok. El proyecto sigue la siguiente estructura de paquetes:

Tabla 11: Estructura de paquetes del proyecto banco-distribuido

Paquete	Responsabilidad
ec.uteq.banco	Clase principal BancoApplication
ec.uteq.banco.config	Configuracion de DataSources multiples
ec.uteq.banco.catalogo	Catalogo distribuido
ec.uteq.banco.service	Servicios de negocio (consulta, transferencia)
ec.uteq.banco.controller	API REST
ec.uteq.banco.model	Entidades y DTOs

El archivo de configuracion define las dos fuentes de datos.

```

1 spring:
2   application:
3     name: banco-distribuido
4   datasource:
5     cuenca:
6       url: jdbc:postgresql://localhost:5432/banco_cuenca
7       username: banco
8       password: banco123
9       driver-class-name: org.postgresql.Driver
10    quito:
11      url: jdbc:postgresql://localhost:5433/banco_quito
12      username: banco
13      password: banco123
14      driver-class-name: org.postgresql.Driver
15  server:
16    port: 8080
17  logging:
18    level:
19      ec.uteq.banco: DEBUG

```

Listing 4: src/main/resources/application.yml: configuracion multi-DataSource

La configuracion de los DataSources se realiza programaticamente para que coexistan dos pools HikariCP independientes.

```

1 package ec.uteq.banco.config;
2
3 import com.zaxxer.hikari.HikariDataSource;
4 import org.springframework.boot.context.properties.ConfigurationProperties;
5 import org.springframework.boot.jdbc.DataSourceBuilder;
6 import org.springframework.context.annotation.Bean;
7 import org.springframework.context.annotation.Configuration;
8 import org.springframework.context.annotation.Primary;
9
10 import javax.sql.DataSource;
11
12 @Configuration
13 public class DataSourceConfig {
14
15     @Bean(name = "cuencaDataSource")
16     @Primary
17     @ConfigurationProperties(prefix = "spring.datasource.cuenca")
18     public DataSource cuencaDataSource() {
19         return DataSourceBuilder.create()
20             .type(HikariDataSource.class)
21             .build();
22     }
23
24     @Bean(name = "quitoDataSource")
25     @ConfigurationProperties(prefix = "spring.datasource.quito")
26     public DataSource quitoDataSource() {

```

```

27     return DataSourceBuilder.create()
28         .type(HikariDataSource.class)
29         .build();
30 }
31 }

```

Listing 5: DataSourceConfig.java: configuracion de los dos DataSources

18.4 Paso 3: Catalogo Distribuido

Implementamos el catalogo distribuido como un componente que conoce la distribucion de los fragmentos y resuelve las consultas hacia el nodo correcto.

```

1  package ec.uteq.banco.catalogo;
2
3  import org.springframework.stereotype.Component;
4
5  import java.util.Arrays;
6  import java.util.List;
7  import java.util.Optional;
8
9  @Component
10 public class CatalogoDistribuido {
11
12     public enum Fragmento {
13         CUENTAS_CUENCA("cuencaDataSource", "Cuenca"),
14         CUENTAS_QUITO("quitoDataSource", "Quito");
15
16         private final String dataSourceBean;
17         private final String oficina;
18
19         Fragmento(String dataSourceBean, String oficina) {
20             this.dataSourceBean = dataSourceBean;
21             this.oficina = oficina;
22         }
23
24         public String getDataSourceBean() { return dataSourceBean; }
25         public String getOficina() { return oficina; }
26     }
27
28     public Optional<Fragmento> resolverPorCiudad(String ciudad) {
29         if (ciudad == null) return Optional.empty();
30         return switch (ciudad.toLowerCase()) {
31             case "cuenca", "loja", "machala" -> Optional.of(Fragmento.
32                 CUENTAS_CUENCA);
33             case "quito", "ibarra", "ambato" -> Optional.of(Fragmento.
34                 CUENTAS_QUITO);
35             default -> Optional.empty();
36         };
37     }
38
39     public Optional<Fragmento> resolverPorNumeroCuenta(String numero) {
40         if (numero == null || numero.length() < 2) return Optional.empty();
41         String prefijo = numero.substring(0, 2);
42         return switch (prefijo) {
43             case "22" -> Optional.of(Fragmento.CUENTAS_CUENCA);
44             case "17" -> Optional.of(Fragmento.CUENTAS_QUITO);
45             default -> Optional.empty();
46         };
47     }
48
49     public List<Fragmento> todosLosFragmentos() {
50         return Arrays.asList(Fragmento.values());
51     }
52 }

```

50 }

Listing 6: CatalogoDistribuido.java: gestion de metadatos de fragmentos

18.5 Paso 4: Servicio de Consulta Distribuida

Implementamos el servicio que orquesta consultas usando el catalogo para enrutar la peticion al nodo correcto.

```

1 package ec.uteq.banco.service;
2
3 import ec.uteq.banco.catalogo.CatalogoDistribuido;
4 import ec.uteq.banco.catalogo.CatalogoDistribuido.Fragmento;
5 import org.springframework.beans.factory.annotation.Qualifier;
6 import org.springframework.context.ApplicationContext;
7 import org.springframework.jdbc.core.JdbcTemplate;
8 import org.springframework.stereotype.Service;
9
10 import javax.sql.DataSource;
11 import java.util.ArrayList;
12 import java.util.Comparator;
13 import java.util.List;
14 import java.util.Map;
15 import java.util.Optional;
16
17 @Service
18 public class ConsultaDistribuidaService {
19
20     private final JdbcTemplate jdbcCuenca;
21     private final JdbcTemplate jdbcQuito;
22     private final CatalogoDistribuido catalogo;
23     private final ApplicationContext context;
24
25     public ConsultaDistribuidaService(
26         @Qualifier("cuencaDataSource") DataSource cuencaDs,
27         @Qualifier("quitoDataSource") DataSource quitoDs,
28         CatalogoDistribuido catalogo,
29         ApplicationContext context) {
30         this.jdbcCuenca = new JdbcTemplate(cuencaDs);
31         this.jdbcQuito = new JdbcTemplate(quitoDs);
32         this.catalogo = catalogo;
33         this.context = context;
34     }
35
36     private JdbcTemplate jdbcDe(Fragmento f) {
37         return switch (f) {
38             case CUENTAS_CUENCA -> jdbcCuenca;
39             case CUENTAS_QUITO -> jdbcQuito;
40         };
41     }
42
43     public Map<String, Object> consultarSaldo(String numeroCuenta) {
44         Optional<Fragmento> destino = catalogo.resolverPorNumeroCuenta(
45             numeroCuenta);
46
47         if (destino.isPresent()) {
48             String sql = "SELECT numero, saldo, oficina FROM cuentas WHERE numero
49                 = ?";
50             List<Map<String, Object>> filas = jdbcDe(destino.get())
51                 .queryForList(sql, numeroCuenta);
52             if (!filas.isEmpty()) {
53                 Map<String, Object> fila = filas.get(0);
54                 fila.put("fuente", destino.get().getOficina());
55                 return fila;
56             }
57         }
58     }
59 }

```

```

54     }
55     }
56     return Map.of("error", "Cuenta no encontrada", "numero", numeroCuenta);
57 }
58
59 public List<Map<String, Object>> listarTodosLosClientes() {
60     String sql = "SELECT cedula, nombre, ciudad FROM clientes ORDER BY nombre"
61     ;
62     List<Map<String, Object>> resultado = new ArrayList<>();
63
64     jdbcCuenca.queryForList(sql).forEach(row -> {
65         row.put("nodo", "CUENCA");
66         resultado.add(row);
67     });
68     jdbcQuito.queryForList(sql).forEach(row -> {
69         row.put("nodo", "QUITO");
70         resultado.add(row);
71     });
72     resultado.sort(Comparator.comparing(r -> r.get("nombre").toString()));
73     return resultado;
74 }
75
76 public List<Map<String, Object>> consultarCuentasPorOficina(String oficina) {
77     Optional<Fragmento> destino = catalogo.resolverPorCiudad(oficina);
78     if (destino.isEmpty()) {
79         return List.of(Map.of("error", "Oficina desconocida", "oficina",
80             oficina));
81     }
82     String sql = "SELECT numero, saldo, oficina FROM cuentas ORDER BY numero";
83     return jdbcDe(destino.get()).queryForList(sql);
84 }

```

Listing 7: ConsultaDistribuidaService.java: consulta y union de resultados

18.6 Paso 5: Transferencia Cross-Site con 2PC Manual

Implementamos la operacion mas critica: una transferencia entre cuentas que residen en distintos nodos. Esto requiere una transaccion distribuida con commit en dos fases.

```

1 package ec.uteq.banco.service;
2
3 import ec.uteq.banco.catalogo.CatalogoDistribuido;
4 import ec.uteq.banco.catalogo.CatalogoDistribuido.Fragmento;
5 import org.slf4j.Logger;
6 import org.slf4j.LoggerFactory;
7 import org.springframework.beans.factory.annotation.Qualifier;
8 import org.springframework.stereotype.Service;
9
10 import javax.sql.DataSource;
11 import java.math.BigDecimal;
12 import java.sql.Connection;
13 import java.sql.PreparedStatement;
14 import java.sql.SQLException;
15 import java.util.Map;
16
17 @Service
18 public class TransferenciaService {
19
20     private static final Logger log = LoggerFactory.getLogger(TransferenciaService
21         .class);
22
23     private final DataSource dsCuenca;
24     private final DataSource dsQuito;

```

```

24     private final CatalogoDistribuido catalogo;
25
26     public TransferenciaService(
27         @Qualifier("cuencaDataSource") DataSource dsCuenca,
28         @Qualifier("quitoDataSource") DataSource dsQuito,
29         CatalogoDistribuido catalogo) {
30         this.dsCuenca = dsCuenca;
31         this.dsQuito = dsQuito;
32         this.catalogo = catalogo;
33     }
34
35     private DataSource dsDe(Fragmento f) {
36         return switch (f) {
37             case CUENTAS_CUENCA -> dsCuenca;
38             case CUENTAS_QUITO -> dsQuito;
39         };
40     }
41
42     public Map<String, Object> transferir(String cuentaOrigen,
43                                         String cuentaDestino,
44                                         BigDecimal monto) {
45         if (monto == null || monto.signum() <= 0) {
46             return Map.of("status", "ERROR", "mensaje", "Monto invalido");
47         }
48
49         Fragmento fOrig = catalogo.resolverPorNumeroCuenta(cuentaOrigen)
50             .orElseThrow(() -> new IllegalArgumentException("Cuenta origen no
51                 enrutable"));
52         Fragmento fDest = catalogo.resolverPorNumeroCuenta(cuentaDestino)
53             .orElseThrow(() -> new IllegalArgumentException("Cuenta destino no
54                 enrutable"));
55
56         Connection cOrig = null;
57         Connection cDest = null;
58         try {
59             cOrig = dsDe(fOrig).getConnection();
60             cDest = dsDe(fDest).getConnection();
61             cOrig.setAutoCommit(false);
62             cDest.setAutoCommit(false);
63
64             // FASE 1 - PREPARACION: debitar origen y acreditar destino
65             if (!debitar(cOrig, cuentaOrigen, monto)) {
66                 rollbackAmbos(cOrig, cDest);
67                 return Map.of("status", "ABORT", "mensaje", "Saldo insuficiente en
68                     origen");
69             }
70             if (!acreditar(cDest, cuentaDestino, monto)) {
71                 rollbackAmbos(cOrig, cDest);
72                 return Map.of("status", "ABORT", "mensaje", "Cuenta destino
73                     inexistente");
74             }
75             registrarTransaccion(cOrig, cuentaOrigen, cuentaDestino, monto);
76
77             // FASE 2 - COMMIT: ambos confirman
78             cOrig.commit();
79             cDest.commit();
80
81             log.info("[2PC] Transferencia OK: {} -> {} por {}",
82                 cuentaOrigen, cuentaDestino, monto);
83             return Map.of("status", "COMMIT",
84                 "origen", cuentaOrigen,
85                 "destino", cuentaDestino,
86                 "monto", monto);

```

```

84     } catch (SQLException ex) {
85         log.error("[2PC] Fallo durante transaccion distribuida", ex);
86         rollbackAmbos(cOrig, cDest);
87         return Map.of("status", "ABORT", "mensaje", ex.getMessage());
88     } finally {
89         cerrar(cOrig);
90         cerrar(cDest);
91     }
92 }
93
94 private boolean debitar(Connection con, String cuenta, BigDecimal monto)
95     throws SQLException {
96     String sql = "UPDATE cuentas SET saldo = saldo - ? WHERE numero = ? AND
97         saldo >= ?";
98     try (PreparedStatement ps = con.prepareStatement(sql)) {
99         ps.setBigDecimal(1, monto);
100        ps.setString(2, cuenta);
101        ps.setBigDecimal(3, monto);
102        return ps.executeUpdate() == 1;
103    }
104 }
105
106 private boolean acreditar(Connection con, String cuenta, BigDecimal monto)
107     throws SQLException {
108     String sql = "UPDATE cuentas SET saldo = saldo + ? WHERE numero = ?";
109     try (PreparedStatement ps = con.prepareStatement(sql)) {
110         ps.setBigDecimal(1, monto);
111         ps.setString(2, cuenta);
112         return ps.executeUpdate() == 1;
113     }
114 }
115
116 private void registrarTransaccion(Connection con, String origen, String
117     destino, BigDecimal monto)
118     throws SQLException {
119     String sql = "INSERT INTO transacciones (cuenta_origen, cuenta_destino,
120         monto, estado) "
121         + "VALUES (?, ?, ?, 'COMPLETADA')";
122     try (PreparedStatement ps = con.prepareStatement(sql)) {
123         ps.setString(1, origen);
124         ps.setString(2, destino);
125         ps.setBigDecimal(3, monto);
126         ps.executeUpdate();
127     }
128 }
129
130 private void rollbackAmbos(Connection c1, Connection c2) {
131     try { if (c1 != null) c1.rollback(); } catch (SQLException ignored) {}
132     try { if (c2 != null) c2.rollback(); } catch (SQLException ignored) {}
133 }
134
135 private void cerrar(Connection c) {
136     try { if (c != null) c.close(); } catch (SQLException ignored) {}
137 }
138 }

```

Listing 8: TransferenciaService.java: simulacion didactica de 2PC en capa de aplicacion

Nota didactica

La implementacion del 2PC mostrada es **didactica**, no apta para produccion. Un 2PC real exige: persistencia del log del coordinador en disco **antes** de comunicar la decision, manejo de timeouts con reintentos idempotentes, identificadores de transaccion globales (XID), y un protocolo

de recuperacion ante caída del coordinador. Java EE provee **JTA** (Java Transaction API) y servidores como Narayana, Bitronix o Atomikos implementan estos detalles. Spring soporta JTA con `@Transactional(transactionManager = "jtaTransactionManager")` sobre estos coordinadores.

18.7 Paso 6: API REST

Exponemos los servicios mediante una API REST.

```

1 package ec.utec.banco.controller;
2
3 import ec.utec.banco.service.ConsultaDistribuidaService;
4 import ec.utec.banco.service.TransferenciaService;
5 import org.springframework.http.ResponseEntity;
6 import org.springframework.web.bind.annotation.*;
7
8 import java.math.BigDecimal;
9 import java.util.List;
10 import java.util.Map;
11
12 @RestController
13 @RequestMapping("/api/banco")
14 public class BancoController {
15
16     private final ConsultaDistribuidaService consulta;
17     private final TransferenciaService transferencia;
18
19     public BancoController(ConsultaDistribuidaService consulta,
20                           TransferenciaService transferencia) {
21         this.consulta = consulta;
22         this.transferencia = transferencia;
23     }
24
25     @GetMapping("/saldo/{numero}")
26     public ResponseEntity<Map<String, Object>> saldo(@PathVariable String numero)
27     {
28         return ResponseEntity.ok(consulta.consultarSaldo(numero));
29     }
30
31     @GetMapping("/clientes")
32     public ResponseEntity<List<Map<String, Object>>> clientes() {
33         return ResponseEntity.ok(consulta.listarTodosLosClientes());
34     }
35
36     @GetMapping("/cuentas/{oficina}")
37     public ResponseEntity<List<Map<String, Object>>> cuentas(@PathVariable String
38         oficina) {
39         return ResponseEntity.ok(consulta.consultarCuentasPorOficina(oficina));
40     }
41
42     @PostMapping("/transferencia")
43     public ResponseEntity<Map<String, Object>> transferir(
44         @RequestParam String origen,
45         @RequestParam String destino,
46         @RequestParam BigDecimal monto) {
47         Map<String, Object> resultado = transferencia.transferir(origen, destino,
48             monto);
49         if ("ABORT".equals(resultado.get("status")) || "ERROR".equals(resultado.
50             get("status"))) {
51             return ResponseEntity.badRequest().body(resultado);
52         }
53         return ResponseEntity.ok(resultado);
54     }
55 }

```


51 }

Listing 9: BancoController.java: endpoints de consulta y transferencia

18.8 Paso 7: Clase Principal Spring Boot

```

1 package ec.uteq.banco;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration;
6
7 @SpringBootApplication(exclude = { DataSourceAutoConfiguration.class })
8 public class BancoApplication {
9
10     public static void main(String[] args) {
11         SpringApplication.run(BancoApplication.class, args);
12     }
13 }

```

Listing 10: BancoApplication.java: punto de entrada de la aplicacion

18.9 Paso 8: Maven pom.xml

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0">
3   <modelVersion>4.0.0</modelVersion>
4   <parent>
5     <groupId>org.springframework.boot</groupId>
6     <artifactId>spring-boot-starter-parent</artifactId>
7     <version>3.3.4</version>
8   </parent>
9   <groupId>ec.uteq</groupId>
10  <artifactId>banco-distribuido</artifactId>
11  <version>1.0.0</version>
12  <properties>
13    <java.version>21</java.version>
14  </properties>
15  <dependencies>
16    <dependency>
17      <groupId>org.springframework.boot</groupId>
18      <artifactId>spring-boot-starter-web</artifactId>
19    </dependency>
20    <dependency>
21      <groupId>org.springframework.boot</groupId>
22      <artifactId>spring-boot-starter-jdbc</artifactId>
23    </dependency>
24    <dependency>
25      <groupId>org.postgresql</groupId>
26      <artifactId>postgresql</artifactId>
27      <scope>runtime</scope>
28    </dependency>
29    <dependency>
30      <groupId>org.springframework.boot</groupId>
31      <artifactId>spring-boot-starter-test</artifactId>
32      <scope>test</scope>
33    </dependency>
34  </dependencies>
35  <build>
36    <plugins>
37      <plugin>

```

```

38     <groupId>org.springframework.boot</groupId>
39     <artifactId>spring-boot-maven-plugin</artifactId>
40   </plugin>
41 </plugins>
42 </build>
43 </project>

```

Listing 11: pom.xml: dependencias Maven del proyecto

18.10 Paso 9: Ejecucion y Pruebas

Practica IDE – Ejecucion del proyecto

Paso 1. Levantar los contenedores PostgreSQL:

```
docker compose -f docker-compose-bdd.yml up -d
```

Paso 2. Verificar el estado:

```
docker compose ps -- ambos en estado healthy
```

Paso 3. Compilar y ejecutar la aplicacion Spring Boot:

```
./mvnw spring-boot:run
```

Paso 4. Probar los endpoints con curl o Postman:

```
curl http://localhost:8080/api/banco/saldo/2201000001
```

```
curl http://localhost:8080/api/banco/saldo/1701000001
```

```
curl http://localhost:8080/api/banco/clientes
```

```
curl http://localhost:8080/api/banco/cuentas/Cuenca
```

```
curl -X POST "http://localhost:8080/api/banco/transferencia?origen=2201000001&destino=1701000001"
```

Paso 5. Verificar la atomicidad de la transferencia consultando ambas cuentas tras la operacion y comprobando el registro en la tabla **transacciones** del nodo origen.

Paso 6. Probar el escenario de falla: detener `docker stop pg_quito` y ejecutar de nuevo la transferencia. Debe responder **ABORT** y los saldos deben permanecer inalterados (rollback exitoso en origen).

18.11 Resultados Esperados

Tras ejecutar las pruebas se observan los siguientes comportamientos clave:

- Las consultas de saldo se dirigen al nodo correcto segun el prefijo del numero de cuenta, demostrando la **transparencia de fragmentacion**.
- Las consultas globales (`/api/banco/clientes`) realizan union distribuida sin que el cliente perciba que los datos provienen de dos nodos.
- Las transferencias cross-site se confirman atomicamente: ambos saldos se actualizan o ninguno (atomicidad).
- Ante caida de un nodo, las transferencias cross-site fallan con rollback limpio, mientras que las operaciones locales en el nodo superviviente siguen funcionando (**autonomia local**).

18.12 Ejercicio Reto: Extension a CockroachDB

Como extension de este ejercicio, los estudiantes pueden migrar el mismo esquema a un cluster CockroachDB de tres nodos, comparando: 1) la complejidad del despliegue, 2) el comportamiento ante caida de un nodo (Raft sigue operando con dos de tres), 3) la transparencia de la fragmentacion automatica (no se requiere el catalogo manual), y 4) las garantias de serializabilidad estricta de CockroachDB versus el aislamiento de PostgreSQL local. Este ejercicio se desarrolla completamente en la **Guia de Practica Experimental GA-SUM-03** de la asignatura.

19. Conclusiones

Las bases de datos distribuidas constituyen una respuesta tecnologica a las exigencias contemporaneas de escala, disponibilidad y dispersion geografica. Mas que una evolucion del SGBD centralizado, son un cambio de paradigma que introduce trade-offs ineludibles entre consistencia, disponibilidad y tolerancia a particiones.

El diseno de una BDD comienza con tres decisiones estrategicas: la arquitectura (cliente-servidor, federada o P2P), el esquema de distribucion (fragmentacion horizontal, vertical, mixta y nivel de replicacion) y el modelo de consistencia (CP o AP, fuerte o eventual). Estas decisiones se sustentan en los requisitos de negocio y se materializan en la eleccion tecnologica (PostgreSQL distribuido, CockroachDB, Cassandra, MongoDB, entre otras).

El control de concurrencia distribuido (2PL, MVCC), los protocolos de commit (2PC, 3PC) y los algoritmos de consenso (Raft, Paxos) son los mecanismos que sostienen las garantias ACID en entornos donde la red puede fallar. El ingeniero de software debe comprender sus principios, sus limitaciones (por ejemplo, el bloqueo del 2PC ante fallo del coordinador) y sus alternativas modernas.

El ejercicio practico del Banco del Austro ilustra que los conceptos teoricos se materializan en codigo Java compilable, contenedores Docker funcionales y comportamientos verificables. La distancia entre la pizarra y la produccion es mas corta de lo que parece, pero exige rigor en el diseno y conciencia plena de los compromisos asumidos.

Referencias

Referencias

- [1] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*, 4th ed. Cham, Switzerland: Springer, 2020. doi: 10.1007/978-3-030-26253-2.
- [2] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017. ISBN: 978-1-4493-7332-0.
- [3] E. Brewer, "CAP twelve years later: How the 'rules' have changed," *IEEE Computer*, vol. 45, no. 2, pp. 23–29, Feb. 2012. doi: 10.1109/MC.2012.37.
- [4] D. J. Abadi, "Consistency tradeoffs in modern distributed database system design: CAP is only part of the story," *IEEE Computer*, vol. 45, no. 2, pp. 37–42, Feb. 2012. doi: 10.1109/MC.2012.33.
- [5] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. 2014 USENIX Annual Technical Conference (USENIX ATC '14)*, Philadelphia, PA, USA, Jun. 2014, pp. 305–319. ISBN: 978-1-931971-10-2.
- [6] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 4th ed., ver. 4.03. Maastricht, The Netherlands: Maarten van Steen, Jan. 2025. ISBN: 978-90-815406-3-6.

- [7] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed. Boston, MA, USA: Addison-Wesley, 2012. ISBN: 978-0-13-214301-1.